

Presentation Model

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 2

Who this book is for . . . 2

A book that stands on its own . . . 2

What you need . . . 3

Conventions . . . 3

I: The Why . . . 4

1. Presentations as data . . . 5

1.1. The problem of N by N . . . 5

1.2. Two goals . . . 5

1.3. Fidelity is not the goal . . . 5

2. A slide is a diagram . . . 7

2.1. What a deck adds . . . 7

2.2. Notes carry annotations . . . 8

2.3. What a typed model gives . . . 8

3. Small on purpose . . . 9

3.1. A contract for generators . . . 9

4. The landscape . . . 10

II: The How . . . 11

5. Getting started . . . 12

5.1. Adding the model . . . 12

5.2. Names that clash . . . 13

5.3. What the build does . . . 13

6. The model . . . 14

6.1. What comes from the diagram model . . . 14

6.2. Presentation . . . 14

6.3. Slide and MasterSlide . . . 15

6.4. Shapes . . . 15

6.5. Text and tables . . . 15

6.6. Connectors . . . 15

7. A deck . . . 16

7.1. Building . . . 16

7.2. Saving and loading . . . 18

7.3. Every word of the deck . . . 18

7.4. Topology . . . 19
III: Looking Ahead . . . **20**
8. Looking ahead . . . **21**
8.1. Loaders and generators . . . 21
8.2. The notes convention . . . 21
8.3. Applications . . . 21
Appendix A: Reference . . . **23**
A.1. From the diagram model . . . 23
Appendix B: Building this book . . . **24**
Appendix C: Glossary . . . **25**

Colophon

Presentation Model: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the presentation model of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime.models.presentation:model:2026.11.0`, namespace URI `runtime.sdk.nasdanika.org/models/presentation`. Every listing in it is included from the module's tests, which are compiled and run when the book is built.

Copyright © 2026 Pavel Vlasov.

The text of this book is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Presentation Model: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

PowerPoint, Visio and Excel are trademarks of Microsoft Corporation. draw.io is a trademark of JGraph Ltd. Eclipse, EMF and ELK are trademarks of the Eclipse Foundation. Java is a trademark of Oracle and its affiliates. Other names may be trademarks of their owners. Their use here identifies the technologies the model relates to and implies no endorsement.

The source of this book, its tests and the model it describes are at github.com/Nasdanika-SDK/runtime, in `models/presentation/model`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about presentations as data: a deck, the slides in it, the shapes on them, the text inside the shapes, the connectors between them, the masters the slides inherit from, and the speaker notes that nothing can query today, as an EMF model. The model stands on the runtime's diagram model and makes one decision that everything else follows from: a slide is a diagram.

The first part of the book is the why: why decks should be data, what follows from a slide being a diagram, what the model leaves out on purpose, and where it stands among the tools for presentations. The second part is the how, for Java developers: a deck built in Java, saved, loaded, and asked for its words and its topology. The third says what comes next: loaders and generators for the formats decks live in.

This edition is the model alone. Loading a deck from PowerPoint, Visio or Draw.io, and writing one, are **PLANNED**.

Who this book is for

Architects and technical writers whose systems are described in decks, and who want those decks to be queried, compared and regenerated, not opened one by one.

Developers who generate presentations from models, or who move a library of drawings from one format to another.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. *Diagram Model*, the book of the model this one stands on, is close to it; you do not need it, or any other book of the series, to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The model's Java module is `org.nasdanika.sdk.runtime.models.presentation`, its Maven coordinates are `org.nasdanika.sdk.runtime.models.presentation:model:2026.11.0`, and its namespace URI is `runtime.sdk.nasdanika.org/models/presentation`.
- Some familiarity with EMF helps in part two: an EMF model is a set of Java interfaces generated from a metamodel, with containment, references, reflection and persistence for free.

Conventions

- **Every listing is included from a test** of the module and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Status labels** mark what this edition does not have yet: PLANNED for designed but not built, SKETCH for partly built.
- **Voice.** I write in the first person.



> Editor's notes record inconsistencies between the model and its documentation found while writing. They are shown in builds for the author and left out of published editions.
>
> This book replaces the model's readme, `models/presentation/model/doc/readme.md`, which is deleted. The readme described the model in Nasdanika's model collection: a model tower with floors (C4, architecture, threat, work, governance, seal, IAM, role, nxcore), sibling models for Visio, PowerPoint, PlantUML, ODP, PDF and Sprotty, `tags` and `properties` on diagram elements, the Markdown model, and a figure, `presentation.drawio`, that is not in the module. The runtime's diagram elements have the core model's `attributes` instead of `tags` and `properties`; Visio, PowerPoint and Excel models were postponed on 2026-10-25, and the roadmap puts the office formats on schema models. This book describes the model as it is, and the rest as planned.
>
> The comment on `ImageShape.location` in the Xcore has a character that is not UTF-8, probably an em dash saved in another encoding.

Part I: The Why

Why a deck should be data, why a slide is a diagram, what the model leaves out on purpose, and where it stands among the tools for presentations. No code is needed to read this part.

Chapter 1. Presentations as data

Decks are where enterprise knowledge goes to stop being data. The architecture that governs a program, the migration plan, the comparison of vendors, the operating model: each exists as a file on a shared drive, authoritative and opaque. Nothing can ask which systems a deck mentions, whether the three decks describing the same platform agree, or which of them still refer to a component that was retired last year. The format is not the obstacle; the absence of a model is.

1.1. The problem of N by N

Formats of presentations are far apart. PowerPoint has its own units, placeholders and DrawingML; Visio has masters, stencils and shape data; PlantUML has no coordinates at all; PDF cannot be edited; Draw.io has style strings. Converters between pairs of them are written by different people, lose different things, and cannot be audited, because the state in the middle is a parse tree that exists for a few milliseconds.

With a model in the middle, N formats need N loaders and N generators, not N times N converters. And the middle is a real EMF resource: saved as XMI, JSON or YAML, compared, queried, and inspected when a conversion goes wrong.

1.2. Two goals

Generation that does not depend on a format. Produce a deck once, from a template, from a mapping or from a language model, and emit PowerPoint for the meeting, PDF for the archive, Draw.io for the wiki, a web page for the site.

Interchange. PlantUML to Draw.io or PowerPoint, and the case enterprises have, at scale, right now: Visio to Draw.io.

1.3. Fidelity is not the goal

That is worth stating plainly, because it is the question every tool of conversion is judged by, and usually the wrong one. A Visio drawing converted through the model will not be identical to the pixel in Draw.io, and a generated deck will not look like the one a designer would have made.

What is preserved is what carries meaning: structure, text, topology, geometry, identity, documentation, and what each shape depicts. What is dropped is what a different tool would have rendered differently anyway.

The wager is that a team moving ten thousand Visio drawings cares about the ten thousand, not about the drop shadow on shape 14, and that a deck whose contents can be queried, compared and regenerated is worth more than a deck that is beautiful and inert. Teams that need fidelity to the pixel should keep the tool that produced the pixels; this model is for when the picture has to become data first.

Chapter 2. A slide is a diagram

Admitting that a slide is a diagram is the entire design. Everything a deck does that a picture cannot, connect two boxes, nest a group in a group, attach an arrow to a shape, say which element of an architecture a box stands for, the runtime's diagram model already does.

So the presentation model does not invent a parallel hierarchy:

- `Presentation` and `Slide` extend the diagram model's `Diagram`;
- `Shape` extends `Node`, with bounds and ports;
- `ConnectorShape` extends `Connection`, with a source, a target and waypoints.

Whatever works on diagrams works on slide 7 of a quarterly review without a line of code specific to presentations: what elements depict, conversion, layout, comparison, and the model tooling of the runtime. The presentation model adds only what a deck has and a diagram does not.

2.1. What a deck adds

Geometry of the deck. A `Presentation` has a `width` and a `height` in points. Points, not the units of one format or pixels, because a neutral space of coordinates is the condition for a round trip that does not scramble the layout.

Text as structure. A `TextShape` contains paragraphs, and a paragraph contains runs of text with their font and their link. The words of a slide are a tree that can be queried, not a string with markup in it.

Masters, which are slides. `MasterSlide` extends `Slide`, so a master is walked, queried, mapped and generated by the same code as a slide. Rebranding an estate of decks is then a transformation over masters, not a person opening files, and a corporate template is a model that a generator targets, not a file it imitates.

Notes. `Notes` are the one thing every format of presentations has and no format of diagrams does.

2.2. Notes carry annotations

Speaker notes are also the most useful carrier of annotations in the family **PLANNED**. By convention, a YAML or JSON block in the notes is read on load into the slide's attributes and what it depicts, and written back on save. Every format has a field for notes, it is invisible when presenting, and it survives a round trip through tools that know nothing about the model. A deck can be annotated in PowerPoint by someone who has never heard of EMF, and come back as a mapped model.

2.3. What a typed model gives

What a shape depicts. `semanticElements`, from the diagram model, lets a box on a strategy deck point at the element of an architecture model it depicts. "Which slides mention this system" becomes a query, not a search of a file share. The link is optional: a shape that depicts nothing is still a documented, addressable element.

Text as a tree. Extract every bullet of a deck, translate a presentation by walking its runs and writing them back, change the font of an estate, feed the text into a search index, or check that no slide uses a term the legal team retired.

Connectors that are connections. A connector has a source and a target, so a slide has topology. That is what makes a deck convertible to a notation like PlantUML or Mermaid at all: without topology, a slide is a bag of boxes near each other.

The mechanics of a round trip. `order` for the order of drawing, `rotation`, `flipHorizontal` and `flipVertical`, rows and cells with `columnSpan` and `rowSpan`: the unglamorous attributes that decide whether a converted deck opens correctly or scrambled. `hidden` on a slide is kept because it carries intent: the backup slides, the appendix, the version for the other audience.

Extension without changing the model. The diagram model's `attributes`, from the runtime's core model, absorb what a format carries and the model does not: an index of a PowerPoint placeholder, a row of Visio shape data, a Draw.io style. Supporting a new format is the work of a loader and a generator, not a release of the model.

Provenance. Every element has markers, so a shape traces back to the file, page, branch and commit it came from, which makes a migration of thousands of files reviewable.

Chapter 3. Small on purpose

There are no animations, transitions, themes, gradient fills, SmartArt, bevels or timings of slides in this model, and there will not be.

There are two reasons. Those things do not survive conversion anyway: a PowerPoint morph transition has no equivalent in Visio or PlantUML, so modeling it neutrally is modeling a fiction. And a small model is what makes generation tractable: a template author, a mapping or a language model can hold fifteen classes in view and produce something structurally valid, which is not true of the formats themselves.

What is rich and specific to a format belongs in a model of that format, such as the runtime's Draw.io model, and in attributes for everything else.

3.1. A contract for generators

A small typed model is a far better contract for a language model than the XML of a format: it fits in a prompt, its meaning is documented, and the output can be validated before anything is drawn. Is it structurally sound? Does every shape depict an element that exists in an approved model? Does any slide refer to a retired system?

The loop becomes: model in, model out, validate, then render to whatever format the audience wants. Review happens on a comparison of models, not by clicking through slides. The distinction is between generating a deck and generating a model that a deck is one rendering of, and the second can be checked before anybody sees it.

Chapter 4. The landscape

Libraries of formats: Apache POI, the Open XML SDK, python-pptx, Aspose.Slides, the ODF Toolkit, PDFBox. These are the sources, not the rivals: loaders and generators are built on libraries like them, or on models of the formats' schemas. Each gives a tree of objects shaped like its format: excellent for writing one file, of no use as a place to keep an estate of presentations, and with no notion of the element a shape stands for.

Services of conversion and office suites without a screen: CloudConvert, GroupDocs, LibreOffice's `--convert-to`, the exporters of Visio, the import of Visio files into Draw.io. They are the pragmatic incumbent for interchange. They are pairwise, one-way and opaque: there is nothing in the middle to inspect, correct, validate, enrich or version, so a conversion that loses shape data loses it silently. And what they lose first is usually what mattered, Visio shape data, Draw.io properties, notes, because it is invisible in the output.

Decks as code: Reveal.js, Marp, Slidev, remark, LaTeX Beamer, Quarto, Pandoc. They are text first, versioned, and good at producing HTML or PDF from Markdown. The source is the artifact, with no graph of objects to query and no stable identity of a shape, and the output is terminal: a business audience asks for a deck it can edit. They make good input formats for a model.

Diagrams as code: PlantUML, Mermaid, D2, Graphviz, the Structurizr DSL. Sources, not competitors. The interesting direction is the one they cannot take themselves: a PlantUML sequence diagram arriving in a deck as editable shapes and connectors, not as an embedded picture.

Software as a service and generators of decks with AI: Canva, Pitch, Gamma, Copilot in PowerPoint, Google Slides with Gemini. The fastest way to get a deck, and they produce no data: nothing to validate the result against, no comparison of this quarter's deck with the last beyond reading both, and no path from the deck to any other tool.

Standards. The OMG has a Diagram Definition specification and interchange packages per notation; there is nothing for presentations. The deck, the most widely produced structured visual artifact in the enterprise, has never had a model for interchange.

The actual competitor: a deck on a shared drive, a library of Visio drawings that one retiring architect can read, and a slide rebuilt by hand every time the audience changes.

Part II: The How

The model in Java: adding it to a project, its classes, and a deck built, saved, loaded and queried.

Chapter 5. Getting started

The model is an EMF model, defined in Xcore and generated into Java when the runtime is built.

5.1. Adding the model

The runtime's models have a bill of materials, `org.nasdanika.sdk.runtime.models:bom`, which manages their versions. Import it in `dependencyManagement`, and depend on the model without a version:

```
<dependency>
    <groupId>org.nasdanika.sdk.runtime.models.presentation</groupId>
    <artifactId>model</artifactId>
</dependency>
```

The model is a named Java module:

Listing 1. The module declaration of the presentation model

```
module org.nasdanika.sdk.runtime.models.presentation {

    exports org.nasdanika.sdk.runtime.models.presentation;
    exports org.nasdanika.sdk.runtime.models.presentation.impl;
    exports org.nasdanika.sdk.runtime.models.presentation.util;

    requires org.nasdanika.sdk.runtime.common;
    requires transitive org.nasdanika.sdk.runtime.models.diagram;

    provides EPackage with PresentationEPackageProvider;

}
```

It requires the runtime's diagram model transitively, which brings the core model and EMF, and the runtime's `common` module, whose derived lists implement `slides` and `masterSlides`. It provides its `EPackage` as a service, so a resource set created with `NasdanikaResourceSet.createAndConfigure()` loads and saves decks in every format the runtime supports.

5.2. Names that clash

`Paragraph` is also the name of a class of the runtime's HTML model, and `Shape`, `Row` and `Cell` are common names. A file that uses both models imports one of them by its full name.

5.3. What the build does

The Xcore definition is in `src/main/resources/org/nasdanika/sdk/runtime/models/presentation/presentation.xcore`. Maven generates the Java sources into `src-gen` with the Xtext Maven plugin. The generated API is in three packages: the interfaces in `org.nasdanika.sdk.runtime.models.presentation`, their implementations in `impl`, and the switch and adapter factory in `util`. The package provider is hand-written, in `services`.

Chapter 6. The model

Area	Classes
Deck	`Presentation`, a diagram: `width` and `height` in points, `slides` and `masterSlides`
Slides	`Slide`, a diagram: `master`, `hidden`, `notes`; `MasterSlide`, a slide; `Notes`, with `text`
Shapes	`Shape`, an abstract diagram node: `order`, `rotation`, `flipHorizontal`, `flipVertical`; `GroupShape`, `ImageShape`, `MediaShape`
Text	`TextShape`, with `paragraphs`; `Paragraph`, with `runs`, `indentLevel` and `alignment`; `TextRun`, with `text`, `fontFamily`, `fontSize`, `bold`, `italic`, `underline`, `color` and `hyperlink`
Tables	`TableShape`, with `rows`; `Row`, with `cells` and a `height`; `Cell`, with `paragraphs`, `columnSpan` and `rowSpan`
Connectors	`ConnectorShape`, a diagram connection: `source`, `target`, `waypoints`

6.1. What comes from the diagram model

Every element of a deck but the text, the tables' rows and cells, and the notes is a diagram element, so it has:

- `id` and `name`, documentation, markers that record where it came from, an `icon` and `uris`, from the runtime's core model;
- `attributes`, a map of strings, for what a format carries and the model does not;
- `semanticElements`, the objects of other models that it depicts;
- `children`, the elements it contains, keyed by `id`.

A shape has `bounds`, relative to its parent, and `ports`; connectable elements know their `incomingConnections` and `outgoingConnections`, kept in step with the connections' `source` and `target`.

6.2. Presentation

A `Presentation` is a diagram whose children are its slides and masters. `slides` and `masterSlides` are derived from `children`: the children that are slides, and the children that are masters. They are views, and changes to them write through to `children`.



`slides` is every `Slide` among the children, masters included, since `MasterSlide` extends `Slide`; a TODO in the Xcore says to exclude masters. Until it does, code that wants the slides that are presented filters masters out, as Chapter 7 does, and `PresentationBookTests.slidesIncludeMasters` records the behavior.

6.3. Slide and MasterSlide

A `Slide` has its `master`, `hidden` for slides kept but not presented, and `notes`. Its shapes and connectors are its children. A `MasterSlide` is a slide, whose shapes appear on the slides that use it.

6.4. Shapes

`Shape` is abstract. `order` is the order of drawing, `rotation` is in degrees, and the flips are booleans. A `GroupShape` groups the shapes that are its children; an `ImageShape` and a `MediaShape` have a `location`, a URI resolved against the resource or the name of an embedded part, and a `contentType`.

6.5. Text and tables

A `TextShape` contains paragraphs; a `Paragraph` contains runs, with an `indentLevel` for bullets and an `alignment`. A `TextRun` is text with the same formatting: `fontFamily`, `fontSize` in points, `bold`, `italic`, `underline`, a `color` as `#RRGGBB`, and a `hyperlink`, which is what makes a generated deck navigable.

A `TableShape` contains rows of cells, and a `Cell` contains paragraphs, as a text shape does, and spans columns and rows.

6.6. Connectors

A `ConnectorShape` is a diagram connection, with a `source` and a `target`, which can be shapes or their ports, and `waypoints` that keep its route.

Chapter 7. A deck

The deck of this chapter is a quarterly review of an architecture: a corporate master with a footer, a slide with two shapes and a connector between them, with notes, and a hidden backup slide with a table.

7.1. Building

Text shapes are built with a small helper:

Listing 2. Text shapes (Deck.java)

```
static TextRun run(String text) {
    TextRun run = PRESENTATION.createTextRun();
    run.setText(text);
    return run;
}

static TextShape text(
        String id,
        double x,
        double y,
        double width,
        double height,
        TextRun... runs) {
    TextShape shape = PRESENTATION.createTextShape();
    shape.setId(id);
    shape.setBounds(bounds(x, y, width, height));                    ①
    Paragraph paragraph = PRESENTATION.createParagraph();
    for (TextRun run: runs) {
        paragraph.getRuns().add(run);                                ②
    }
    shape.getParagraphs().add(paragraph);
    return shape;
}
```

① Bounds come from the diagram model: a position relative to the parent, and a size, in points.

② A paragraph of runs. Each run has its own formatting.

Listing 3. The deck (Deck.java)

```
static Presentation review() {
    Presentation deck = PRESENTATION.createPresentation();
    deck.setName("Quarterly architecture review");
    deck.setWidth(960);                                              ①
    deck.setHeight(540);
```

```
    MasterSlide corporate = PRESENTATION.createMasterSlide();
    corporate.setId("corporate");
    corporate.setName("Corporate");
    corporate.getChildren().add(text("footer", 20, 510, 400, 20, run("Internal")));
    deck.getChildren().add(corporate);                                  ②

    Slide architecture = PRESENTATION.createSlide();
    architecture.setId("architecture");
    architecture.setName("Architecture");
    architecture.setMaster(corporate);
    TextShape web = text("web", 100, 200, 200, 80, run("Web application"));
    TextShape db = text("db", 600, 200, 200, 80, run("Accounts database"));
    ConnectorShape reads = PRESENTATION.createConnectorShape();
    reads.setId("reads");
    reads.setSource(web);                                               ③
    reads.setTarget(db);
    architecture.getChildren().add(web);
    architecture.getChildren().add(db);
    architecture.getChildren().add(reads);
    Notes notes = PRESENTATION.createNotes();
    notes.setText("The database is the system of record for accounts.");
    architecture.setNotes(notes);                                       ④
    deck.getChildren().add(architecture);

    Slide backup = PRESENTATION.createSlide();
    backup.setId("backup");
    backup.setName("Backup: volumes");
    backup.setHidden(true);                                             ⑤
    backup.setMaster(corporate);
    backup.getChildren().add(table("volumes",
            new String[] { "System", "Requests per day" },
            new String[] { "Web application", "1,200,000" }));
    deck.getChildren().add(backup);
    return deck;
}
```

① The size of a slide, in points: 16 by 9.

② Masters and slides are the children of the presentation.

③ A connector is a diagram connection. Setting its source adds it to the shape's outgoing connections.

④ Speaker notes.

⑤ Kept, not presented.

7.2. Saving and loading

A deck is a model, so it is saved and loaded like any other, here as XMI:

Listing 4. Saving and loading a deck (`PresentationBookTests.java`)

```
ResourceSet resourceSet = NasdanikaResourceSet.createAndConfigure();   ①
URI uri = URI.createFileURI(directory.resolve("review.xml").toString());
Resource resource = resourceSet.createResource(uri);
resource.getContents().add(Deck.review());
resource.save(null);

Presentation deck = (Presentation) NasdanikaResourceSet.createAndConfigure()
        .getResource(uri, true)
        .getContents()
        .getFirst();
assertThat(deck.getMasterSlides()).extracting(Slide::getName).containsExactly("Corporate");
```

① A resource set configured from the module path. With the runtime's `emf-json` module there, `review.yml` would be YAML.

7.3. Every word of the deck

Text is a tree, so extracting the words of what is presented is a walk:

Listing 5. The words of the presented slides (`PresentationBookTests.java`)

```
List<String> words = deck.getSlides().stream()
    .filter(slide -> !(slide instanceof MasterSlide))                ①
    .filter(slide -> !slide.isHidden())                              ②
    .flatMap(slide -> slide.getChildren().stream())
    .flatMap(shape -> switch (shape) {
        case TextShape text -> text.getParagraphs().stream();
        case TableShape table -> table.getRows().stream()
                .flatMap(row -> row.getCells().stream())
                .flatMap(cell -> cell.getParagraphs().stream());
        default -> Stream.<Paragraph>empty();
    })
    .flatMap(paragraph -> paragraph.getRuns().stream())
    .map(TextRun::getText)
    .toList();
assertThat(words).containsExactly("Web application", "Accounts database");
```

① Masters are slides too: in this edition `slides` includes them.

② The backup slide, with its table, is not presented.

The same walk, writing runs back, translates a deck or changes its font.

7.4. Topology

Connectors are connections, so a slide has topology to navigate:

Listing 6. What the web application connects to (PresentationBookTests.java)

```
Slide architecture = deck.getSlides().stream()
    .filter(slide -> "architecture".equals(slide.getId()))
    .findFirst()
    .orElseThrow();
TextShape web = (TextShape) architecture.getChildren().getFirst();
List<String> targets = web.getOutgoingConnections().stream()          ①
    .map(Connection::getTarget)
    .map(target -> ((TextShape) target).getParagraphs().getFirst())
    .map(paragraph -> paragraph.getRuns().getFirst().getText())
    .toList();
assertThat(targets).containsExactly("Accounts database");
```

① The diagram model keeps both ends: a shape knows its outgoing and incoming connections.

Part III: Looking Ahead

Loaders and generators for the formats decks live in, and what they make possible.

Chapter 8. Looking ahead

Everything in this chapter is PLANNED.

8.1. Loaders and generators

Each format is a loader and a generator over this model, not a model with ambitions of its own:

- **Draw.io**, through the runtime's Draw.io model, which already reads and writes Draw.io documents: a page per slide.
- **PowerPoint and Visio**, on models of their schemas, PresentationML and DrawingML, and VSDX, rather than on a library of the format; postponed on 2026-10-25.
- **PlantUML**, as an input: diagrams written as text, arriving as editable shapes.
- **PDF**, for the archive, and **web pages**, for sites, generated with the runtime's HTML and SVG models.

8.2. The notes convention

A YAML or JSON block in the notes read on load into attributes and what a slide depicts, and written back on save (Chapter 2).

8.3. Applications

Visio to Draw.io. Enterprises hold large estates of Visio drawings: networks, processes, floor plans, flows of data, maps of integration, accumulated over decades and locked behind licenses that most of the people who need to read them do not have. Through this model the migration is a batch job: load the drawings, keep shape data as attributes and what shapes depict, emit Draw.io, review the result as a comparison of models, and publish the estate as a site that can be searched. What arrives on the other side is the first time the estate can be queried.

One generator, many renderings. A quarterly architecture review generated from a model of the architecture comes out as PowerPoint for the meeting, PDF for the record, Draw.io for the wiki, and a page for the portal: one model, four renderings, and no divergence between them.

Mining the estate of decks. The decks an organization already has document its systems,

programs, vendors and intentions. Load them, extract shapes, text and connectors, and mine them into models of the architecture or the work. Even the shallow result is useful: what is mentioned, how often, by whom, and where the descriptions disagree.

Presentations as documentation sites. Every slide a page, every shape an anchor, notes as the content of the page, connectors as links, and markers pointing back to the file.

One deck, many audiences. The board version, the engineering version, the client version, the version without the numbers: audience becomes a decision of rendering over one model.

Review, comparison and merge. A comparison of models answers "what changed in this deck" with added slides and connections that point elsewhere, which makes a deck reviewable in a pull request. A merge of three versions, the previous generation as the base, the fresh generation and the deck a person edited, keeps a generated deck alive past its second run.

Layout. With bounds, ports, waypoints and topology in a neutral model, ELK lays out the contents of a slide, as it lays out diagrams, which is what makes a generated deck readable instead of a grid of boxes.

Appendix A: Reference

Class	Extends	Features
Presentation	diagram Diagram	width, height, slides (derived), masterSlides (derived)
Slide	diagram Diagram	master, hidden, notes
MasterSlide	Slide	
Notes		text
Shape (abstract)	diagram Node	order, rotation, flipHorizontal, flipVertical
GroupShape	Shape	
TextShape	Shape	paragraphs
Paragraph		runs, indentLevel, alignment
TextRun		text, fontFamily, fontSize, bold, italic, underline, color, hyperlink
ImageShape	Shape	location, contentType
MediaShape	Shape	location, contentType
TableShape	Shape	rows
Row		cells, height
Cell		paragraphs, columnSpan, rowSpan
ConnectorShape	diagram Connection	

A.1. From the diagram model

Class	Features
DiagramElement	semanticElements, children; and from the core model id, name, documentation, markers, icon, uris, attributes
Connectable	incomingConnections, outgoingConnections
Node	bounds, ports
Connection	source, target, waypoints
Point, Bounds	x, y; width, height

Appendix B: Building this book

This book is AsciiDoc in `models/presentation/model/src/docs/asciidoc`, built with the model after its tests have passed:

```
mvn -pl models/presentation/model -am verify                          # the model, its tests and
this book
mvn -pl models/presentation/model verify -Dzephyr.editor-notes=false  # without editor's notes
mvn -pl models/presentation/model verify -Dasciidoctor.skip=true      # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the model's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/models/presentation/tests/book`, and of the module declaration. A listing line holds at most 96 characters, and a warning fails the build.

Appendix C: Glossary

Connector

A line between two shapes, or their ports: a connection of the diagram model.

Deck

A presentation: an ordered set of slides with their masters.

Master

A slide whose shapes appear on the slides that use it, such as a corporate template.

Notes

The text for the speaker of a slide, not shown when presenting; here also a carrier of annotations.

Point

A unit of length of 1/72 of an inch, the unit of the model's geometry.

Run

A piece of text with one formatting, within a paragraph.

Semantic element

An object of another model that a diagram element depicts, such as the system a box on a slide stands for.